

DevOps & CI/CD para Ingeniería de Software

Scripts, Docker Multi-Stage, GitFlows y GitHub Actions

Departamento de TIC · Universidad Icesi

Facultad de Ingeniería · Infraestructura lasLab

23 de agosto de 2026

Ruta de Aprendizaje

- 1 Módulo 1: Automatización con Scripts
- 2 Módulo 2: Contenerización con Docker
- 3 Módulo 3: Flujos de Trabajo en Git (GitFlow)
- 4 Módulo 4: CI/CD con GitHub Actions

Directivas de Ejecución Segura

- `set -e`: Detiene la ejecución ante cualquier error.
- `set -u`: Trata variables nulas como errores.
- `set -o pipefail`: Captura fallos en pipelines (|).

deploy-safe.sh

```
#!/usr/bin/env bash
set -euo pipefail

TARGET="${1:-/home/iaslab/app}"

if [ ! -d "$TARGET" ]; then
    echo "Error: Directorio no existe"
    exit 1
fi

docker compose pull && docker compose up -d
```

Módulo 2: Multi-Stage Builds en Docker

Backend: Spring Boot (Java 17)

```
# 1. Build Stage
FROM eclipse-temurin:17-jdk AS builder
WORKDIR /app
COPY . .
RUN ./gradlew bootJar --no-daemon

# 2. Runtime Stage (< 150MB)
FROM eclipse-temurin:17-jre
USER spring
COPY --from=builder /app/build/*.jar app.jar
EXPOSE 9091
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Frontend: React + Nginx Alpine

```
# 1. Build Stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN npm ci && npm run build

# 2. Runtime Stage (< 25MB)
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Módulo 2: Orquestación y Reverse Proxy Nginx

docker-compose.yml

```
services:
  web:
    image: github-action-test:latest
    container_name: test-app
    restart: unless-stopped
    ports:
      - "127.0.0.1:9088:80"
```

Nginx Host Proxy (/home/iaslab/*/nginx.conf)

```
location = /iaslab/app {
    return 301 /iaslab/app/$is_args$args;
}

location /iaslab/app/ {
    proxy_pass http://127.0.0.1:9088/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Módulo 3: Estrategia de Ramas y SemVer

Rama	Propósito	Versionado SemVer	Ambiente
main	Producción Estable	1.0.0 (Release)	Servidor Producción
release/*	Pruebas QA / Staging	1.0.0-RC1	Servidor grid100
develop	Integración Continua	1.0.0-SNAPSHOT	Local / Dev
feature/*	Nuevas Funcionalidades	N/A	Ramas de Trabajo
hotfix/*	Parches Críticos	1.0.1 (Patch)	Producción Directa

Conventional Commits

```
feat(auth): add JWT login    |    fix(survey): resolve 500 null pointer    |    ci:  
update runner
```

Módulo 4: Workflows Condicionales por Rama

Rama main (Despliegue Estático)

```
on:
  push:
    branches: [ main ]

jobs:
  deploy-static:
    runs-on: [ self-hosted ]
    steps:
      - uses: actions/checkout@v4
      - run: |
          cp index.html /home/iaslab/app/
          sudo nginx -t && sudo systemctl reload
            nginx
```

Rama docker (Contenedores)

```
on:
  push:
    branches: [ docker ]

jobs:
  deploy-docker:
    runs-on: [ self-hosted ]
    steps:
      - uses: actions/checkout@v4
      - run: |
          docker compose build
          docker compose up -d
          sudo systemctl reload nginx
```

Módulo 4: Arquitectura Self-Hosted Runner (grid100)

¿Por qué usar Self-Hosted Runners en lasLab?

- **Seguridad Saliente:** El runner se conecta a GitHub mediante *HTTPS long-polling*; no requiere puertos públicos entrantes abiertos hacia el servidor.
- **Acceso a Recursos Internos:** Ejecuta comandos con acceso nativo al socket de Docker y a las bases de datos locales (PostgreSQL 18).
- **Sin Límites de Minutos:** Ejecuciones ilimitadas en infraestructura institucional.

Monitoreo y URLs Activas

- Portal del Runner: <https://pi2tools.icesi.edu.co/iaslab/github-action/>
- Despliegue Estático: <https://pi2tools.icesi.edu.co/iaslab/github-action-test/>
- Despliegue Docker: <https://pi2tools.icesi.edu.co/iaslab/github-action-docker/>

Universidad Icesi

Departamento de TIC · Ingeniería de Software

¡Preguntas y Espacio de Laboratorio!

<https://pi2tools.icesi.edu.co/iaslab/docs/>